Les Tableaux Unidimensionnels et le Passage par Adresse en Langage C

Thème principal : Structures de données séquentielles, gestion mémoire, pointeurs et Array Decay

Synthèse exécutive : Ce cours traite de la représentation et de la manipulation des tableaux à une dimension en langage C. Il détaille la contiguïté mémoire des éléments alloués sur la pile, la formule de calcul d'adresse garantissant un accès direct en complexité O(1), et le mécanisme fondamental de l'Array Decay (dégradation automatique du nom d'un tableau en un pointeur vers son premier élément lors du passage à une fonction). Le cours formalise la nécessité absolue de passer la taille effective d'un tableau en paramètre séparé et explique le fonctionnement du passage par adresse pour agir directement sur les données originales.

Définition, disposition contiguë en mémoire vive et calcul d'adresse O(1)

Lorsqu'un tableau est déclaré en langage C (par exemple `int tab[5];`), le compilateur réserve un bloc ininterrompu de mémoire vive. La taille totale allouée correspond exactement au produit du nombre d'éléments par la taille du type de donnée (`N * sizeof(type)`). Pour un tableau de 5 entiers de 4 octets, 20 octets consécutifs sont réservés sur la pile. L'accès à un élément via la syntaxe `tab[i]` ne requiert aucun parcours séquentiel des éléments précédents. Le système calcule instantanément l'adresse mémoire exacte de la case visée grâce à la formule d'arithmétique système : `Adresse(tab[i]) = Adresse_Base + (i * sizeof(type))`. L'indice `i` correspond à l'offset (le nombre d'éléments de décalage depuis le début du tableau). C'est pour cette raison exacte que le premier élément possède l'index 0 (décalage nul). Ce calcul d'adresse garantit des performances maximales avec un temps d'accès en complexité algorithmique constante O(1).

- Tableau unidimensionnel : Conteneur statique d'éléments de type identique stockés de manière séquentielle.
- Contiguïté mémoire : Les cases mémoire sont allouées de façon strictement contiguë, sans aucun intervalle libre entre deux éléments.
- Indexation à partir de zéro : Les indices vont de 0 à N-1 pour un tableau de N éléments, l'indice représentant le décalage (offset).
- Complexité O(1) : Accès direct à n'importe quel élément en temps constant par calcul mathématique d'adresse.

Cas concret enseignant : *Le professeur a utilisé la métaphore d'un meuble de rangement vertical comportant des tiroirs rigoureusement identiques et collés les uns au-dessus des autres, numérotés de 0 à N-1. Si vous connaissez le sol (l'adresse de base de la structure) et la hauteur exacte d'un tiroir (`sizeof(type)`), vous calculez immédiatement la hauteur exacte du tiroir `i` sans avoir besoin d'ouvrir ou de mesurer tous les tiroirs du dessous.*

Pointeurs, arithmétique des pointeurs et mécanisme d'Array Decay

En langage C, le passage d'un tableau par valeur (copie intégrale du contenu) est proscrit pour préserver les performances et la mémoire. Dès que le nom d'un tableau est utilisé comme argument dans un appel de fonction ou dans une expression de pointeurs, il subit une dégradation automatique appelée 'Array Decay'. Le nom du tableau s'évalue alors comme un pointeur constant de type `type*` contenant l'adresse du premier élément (`&tab;[0]`). Ce mécanisme présente deux conséquences fondamentales : 1. La fonction appelée reçoit l'adresse de l'original. Toute modification effectuée dans la fonction (`tab[i] = valeur`) altère directement le tableau de la fonction appelante (passage par adresse). 2. Le paramètre formel dans la fonction devient un simple pointeur. Le type d'origine 'tableau de N éléments' est définitivement perdu au sein du scope de la fonction. Par conséquent, la fonction est incapable de deviner le nombre d'éléments alloués. La taille effective du tableau doit être transmise obligatoirement en paramètre complémentaire.

- Array Decay : Conversion implicite et automatique du nom d'un tableau en un pointeur vers son premier élément (`&tab;[0]`) lors de sa transmission.

• Passage par adresse : Transmission de l'emplacement mémoire original évitant la duplication de données et permettant la modification directe.

• Arithmétique des pointeurs : Équivalence stricte entre l'accès indexé `tab[i]` et la déréférenciation de pointeur `*(tab + i)`.

• Perte de dimensionnalité : Impossibilité pour une fonction de déterminer la taille d'un tableau à partir de son seul pointeur.

Cas concret enseignant : *Pour expliquer le passage par adresse et l'Array Decay, l'enseignant a expliqué que transmettre un tableau à une fonction ne revient pas à fabriquer une réplique exacte de sa maison dans le jardin de son ami (passage par valeur), mais plutôt à lui donner les coordonnées GPS précises de sa maison (passage par adresse). L'ami utilise ce repère GPS pour se rendre sur place et peindre directement les murs du bâtiment d'origine.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante-t-il ou affiche-t-il des valeurs absurdes lorsque j'essaie de lire ou d'écrire dans tab[5] alors que mon tableau a été déclaré avec une taille de 5 (int tab[5];) ?
Réponse du professeur : Un tableau de taille 5 indexe ses éléments de 0 à 4. L'accès à `tab[5]` tente d'accéder à la 6ème case mémoire (un décalage de 5 entiers). Le C n'effectue aucun contrôle d'agrandissement ou de débordement d'indice à l'exécution (pas de Boundary Checking). Accéder à `tab[5]` constitue un dépassement de tampon (Buffer Overflow) entraînant un comportement indéterminé (Undefined Behavior) : soit vous lisez une valeur poubelle présente en mémoire, soit vous écrasez une autre variable, soit le système d'exploitation stoppe net le programme avec une erreur de segmentation (Segmentation Fault).

Question étudiant : Est-ce qu'on peut calculer la taille d'un tableau à l'intérieur d'une fonction avec l'expression sizeof(tab) / sizeof(tab[0]) ?
Réponse du professeur : Non, c'est un piège classique ! Dans le scope de la fonction, l'Array Decay a transformé `tab` en un simple pointeur (`int*`). L'expression `sizeof(tab)` renverra la taille du pointeur lui-même (soit 4 ou 8 octets selon l'architecture 32/64 bits) et non la taille du tableau. La division `sizeof(tab) / sizeof(tab[0])` donnera un résultat complètement erroné (souvent 1 ou 2). C'est pourquoi la taille doit impérativement être passée en paramètre entier explicite à la fonction.

■ Exercice & Démonstration d'Application

Énoncé : Écrire une fonction C nommée 'remplir_et_calculer_moyenne' qui prend en paramètre un tableau d'entiers et sa taille effective. La fonction doit demander à l'utilisateur de saisir chaque note au clavier pour remplir le tableau, puis calculer et retourner la moyenne arithmétique sous la forme d'un nombre à virgule (double). Sécuriser la fonction contre la division par zéro.

Résolution pas à pas :
1. Prototypes et types : Définir la fonction `double remplir_et_calculer_moyenne(int tab[], int taille)` ou `double remplir_et_calculer_moyenne(int *tab, int taille)`. 2. Sécurité : Ajouter une condition préalable pour vérifier si `taille <= 0`. Si c'est le cas, retourner `0.0` pour éviter toute division par zéro. 3. Initialisation de l'accumulateur : Déclarer `double somme = 0.0;` afin d'éviter la troncature de la division entière. 4. Parcours et saisie : Utiliser une boucle `for` avec un indice `i` allant de `0` à `taille - 1` pour demander la saisie via `scanf("%d", &tab;[i])` et accumuler le résultat dans `somme`. 5. Calcul et retour : Diviser `somme` par `taille` et retourner la valeur de type `double`.

Résultat : #include double remplir_et_calculer_moyenne(int tab[], int taille) { if (taille <= 0) { return 0.0; } double somme = 0.0; for (int i = 0; i < taille; i++) { printf("Saisir la note [%d] : ", i + 1); scanf("%d", &tab;[i]); somme += tab[i]; } return somme / taille; } // Exemple d'exécution : // Saisies : 12, 14, 16 pour taille = 3 // Calcul : (12 + 14 + 16) / 3 = 42.0 / 3 = 14.000000

■ Auto-Évaluation Flash d'Examen

Q : Quel est l'indice du dernier élément accessible d'un tableau déclaré par 'int t[10];' et que se passe-t-il si l'on tente d'accéder à t[10] ?

R : L'indice du dernier élément est 9. Accéder à t[10] provoque un débordement d'indice (Buffer Overflow) entraînant un comportement indéterminé (Undefined Behavior) ou un SegFault.

Q : Que contient exactement l'identifiant d'un tableau lorsqu'il est utilisé seul sans crochets dans une expression C ?

R : Il contient l'adresse mémoire de son premier élément (&tab;[0]), suite au mécanisme d'Array Decay.

Q : Pourquoi la signature d'une fonction manipulant un tableau C doit-elle obligatoirement inclure sa taille en paramètre ?

R : Parce qu'à la suite de l'Array Decay, le tableau se dégrade en un simple pointeur lors du passage à la fonction. La fonction n'a aucun moyen natif de déterminer la dimension du tableau récepteur.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765